Renewal Risk Worklist

A ranked Monday list for every CSM, an unattended nightly agent, and outreach a human approves



Fello take-home · Staff Engineer, Data Architecture & GTM AI Platform
github.com/saurabhkashyap259/FelloAI

The problem is attention, not visibility

Two renewals were lost while the warning signs sat in support tickets and product usage.

Nobody was watching across five systems, and a CSM can't read five systems every week.

So the goal: every Monday, each CSM sees the few accounts most likely not to renew, why, and how sure the system is, early enough to act.

~120 accounts per CSM

~10 they can act on in a week

5 source systems to read

Where I disagreed with the request

ASKED FOR	BUILT INSTEAD	WHY
Red/yellow/green health dashboard	**A ranked worklist per CSM, with reasons**	A colour doesn't say who to call first. Bands follow CSM capacity, not fixed thresholds
Auto-email customers who turn red	**Drafts a CSM approves and sends. The system sends nothing**	A false positive tells a healthy customer they're "at risk". That's hard to undo
"Nothing slips"	**Hit rate and lead time, measured, with misses reported**	No model catches everything. An unmeasurable promise turns into blame later

How it works

1

Ingest

5 sources, dedupe, identity matching, quarantine for unmatched records

2

Signals

30 point-in-time signals. Nothing from the future leaks in

3

Model

Pooled LightGBM, per-tenant calibration, gated weekly retrain

4

Nightly agent

Score, compare with last night, alert, gather evidence, draft

5

Worklist

Ranked per CSM, with reasons that drill down to source records

Stack Python 3.12 · Postgres 16 with forced row-level security · LightGBM + scikit-learn · FastAPI + HTMX · Claude Sonnet 5.5 for drafting only

Synthetic data with planted answers

5,000

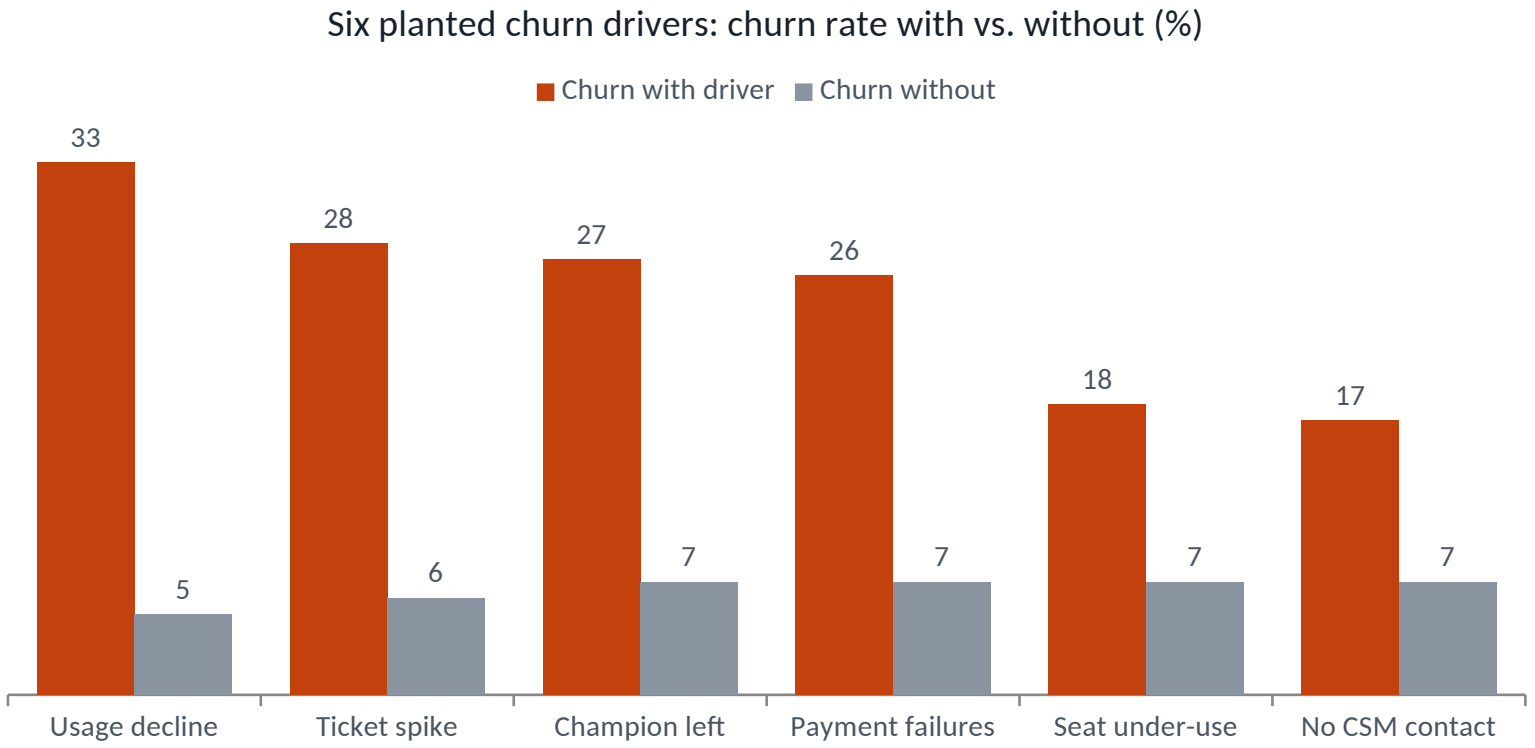
accounts

208,919

events from 5 sources

4

tenants, 2 to 22 CSMs



Mess planted on purpose 2.5% duplicates · ~6% late by 1–10 days · out-of-order delivery · local timestamps without a timezone · ~1% unmatched records · 5% CRM/billing conflicts · 10% sparse accounts

Results on the held-out last six months

Criterion	Target	Result	
Churn concentrated in "Act this week" vs. all accounts	≥3×	**5.5×**	Met
Risk percentages honest (largest calibration gap)	≤10 pts	**6.4 pts**	Met
Silent churners flagged 30+ days ahead	≥70%	**74%**	Met
Nightly run time, ~5,000 accounts	<30 min	**~21 s, 7 of 7 nights**	Met
Alerts per CSM per week	≤5	**0.95**	Met
Churners flagged 60+ days before renewal	≥70%	**26%**	Missed
Planted drivers among the top 10 signals	≥5 of 6	**4 of 6**	Missed
Beats logistic baseline on hit rate and lead time	both	**83% vs 82%; 40.5 vs 42 days**	Partly

Weekly worklists replayed over a later period than training. Pooled LightGBM, promoted over the baseline by the gate. Source: reports/eval_2026-10-26.json

What the misses mean

26%

of churners reached the top of a CSM's list 60+ days before renewal. Target: 70%.

83% reached it at some point. Median lead time: 40 days.

WHY

- The generator starts each driver 55–160 days before renewal. At 60 days out, many haven't started yet.
- Payment failures and champion departure hit ~4% of renewals each: strong effects, too rare to rank high on average.
- LightGBM only marginally beats logistic regression on this data.

WHAT I DID AND WOULD DO

- Froze the generator before evaluating, so the misses are reported, not tuned away.
- Next: a 180-day horizon with lead-time-weighted training, and denser usage data.

The unattended nightly agent



7 of 7

simulated nights succeeded

~21 s

per night, ~5,000 accounts

0.95

alerts per CSM per week

- Runs on a schedule with no human trigger. Running the same night twice does nothing.
- Catches up at most 3 missed nights, then flags the run as needing attention.
- Alerts only on change: entered the top 10, a risk jump, or a severe event near renewal. At most 3 per CSM a night and 10 a week.
- Claude sees only an evidence pack of numbered facts. A validator rejects unknown facts, numbers and dates, and the word "risk" in customer emails. Any failure falls back to a labelled template.

Isolation and traceability

1 Tenant isolation

- Forced row-level security on every tenant table in Postgres
- Web and job roles aren't table owners. No tenant set means zero rows
- Another tenant's or CSM's account returns the same 404 as a made-up ID, and writes an audit row
- A test fails if any route lacks an access rule

2 Every number traceable

- Displayed scores come from immutable rows: run, model version, signal-definition version
- Any score can be recomputed exactly from stored inputs
- Each reason drills down to its definition, window and source records
- The runs page shows what came in, duplicates, quarantined records and alerts raised or suppressed

Skipped on purpose, and what breaks at 10x

Skipped on purpose

- Real connectors (the generator uses the same ingest path)
- Sending email, CRM write-back, Slack alerts
- SSO, sign-up, mobile layouts
- Automated data purging
- Survival models

Breaks first at 10x (50k accounts, 2M events)

1. **Feature computation** loads each tenant's full history into pandas. Move to incremental SQL aggregates
2. **Training** rebuilds every snapshot. Cache them in a feature store
3. **LLM drafts** run inside the nightly transaction. Give them their own queue
4. **Cron + advisory lock** should become an orchestrator (Dagster, Temporal)
5. **Sign-in rate limiter** is in memory, single instance only. Move to Redis
6. **Model files and tenants** share one Postgres. Object storage; separate databases for regulated tenants